

Range Queries

Kanishk Goel

1 June, 2026

1 Range Queries

The task is to calculate a value based on a subarray of an array. A naive way to answer a range query is to iterate over i to j such that $i < j$ and find the value that the query asks for. This runs in $O(n)$ and for q queries will require $O(nq)$ time. Notation - $func_q(a_1, a_2 \dots a_n)$

2 Sum Queries - Prefix Sum Subarray

Range query where value required is simply the sum of the elements in that range. This is solved in $O(n)$ via prefix sum array.

A variant of the same is to generalize the idea in higher dimensions. For example in $2D$, we can require the sum of a rectangle from (i, j) to $(i + l, j + b)$.

The solution to this, is to simply calculate a prefix sum subarray f , which calculates sum of the rectangle from $(0, 0)$ to (i, j) . Now the answer would simply be $f(i, j) + f(i + l, j + b) - f(i, j + b) - f(i + l, j)$.

3 Min Queries - Sparse Table

This is a $O(n \log n)$ preprocessing method, after which we can answer most queries in $O(\log n)$ time and any minimum/maximum query in $O(1)$ time.

3.1 Idea

Every number has a binary representation, therefore precalculate all values of $min_q(a, b)$ where $b - a + 1$ is a power of 2.

Every 2^k range can be calculated by using the previous 2^{k-1} ranges as

$$min_q(a, b) = \min(min_q(a, a + w - 1), min_q(a + w, b)),$$

where $w = (b - a + 1)/2$

Now any query (a, b) can be solved by representing the range $[a, b]$ as $[a, a + k - 1] \cup [b - k + 1, b]$ where k is the largest power of 2 that does not exceed $b - a + 1$.

$$\min_q(a, b) = \min(\min_q(a, a + k - 1), \min_q(b - k + 1, b)),$$

3.2 Implementation

We use a $2D$ array, where $st[i][j]$ stores the answer for the range $[j, j + 2^i - 1]$. Size of the array is $(K + 1) \times MAXN$ where $MAXN$ is the biggest possible array length and $K \geq \lfloor \log_2 MAXN \rfloor$. $MAXN$ dimension is second for cache friendliness.

3.3 Conclusion

Sparse Table is therefore, best used for idempotent functions (functions whose repeated application doesn't change the result beyond the first time).

4 Fenwick Tree